

NAME: ERIC MENJA GICHOHI

EMAIL : mejjaeric6@gmail.com

part 1: Theoretical Understanding (40%)

1. Short Answer Questions

- **Q1:** Explain the primary differences between TensorFlow and PyTorch. When would you choose one over the other?

Both are open-source frameworks for Deep Learning, they differ fundamentally in their development philosophy, computation graphs, and deployment strengths

1. The Key Differences

Computational Graphs (Static vs. Dynamic):

- **PyTorch** uses a **Dynamic Computational Graph** (Define-by-Run). This means the graph is built on the fly as the code is executed. This makes it feel like standard Python; you can use standard Python flow control (loops, if-statements) and debug it easily with a `print()` statement or a debugger.
- **TensorFlow** (historically) used a **Static Computational Graph** (Define-and-Run). You had to define the entire structure of the neural network before running it. *Note: TensorFlow 2.0 introduced "Eager Execution" to mimic PyTorch's dynamic style, but its core strength remains in compiling static graphs (via `@tf.function`) for optimized deployment*

Deployment vs. Research:

- **TensorFlow** is often considered "production-first." Its ecosystem (TFLite, TF Serving, TensorFlow.js) provides a seamless end-to-end pipeline for deploying models to mobile devices, browsers, and large servers.
- **PyTorch** is "research-first." Its flexibility and pythonic nature allow researchers to prototype complex, novel architectures quickly without fighting the framework.

Q2: Describe two use cases for Jupyter Notebooks in AI development.

Jupyter Notebooks are an interactive computing environment that allows developers to mix live code, equations, visualizations, and narrative text. They are ubiquitous in AI for two primary reasons:

1. Exploratory Data Analysis (EDA) and Visualization

Before building a model, an AI engineer must understand the data. Jupyter Notebooks are ideal for this because they allow for **instant feedback**.

- **How it works:** You can write a snippet of code to load a dataset and create a chart (using libraries like Matplotlib or Seaborn). The chart renders *immediately* below the code cell.
- **Why it matters:** If you are analyzing crop yields, you can instantly plot a graph of "Rainfall vs. Yield" to see if there is a correlation. You can visually inspect images of plant leaves to ensure they are labeled correctly before feeding them into a neural network.

2. Incremental Prototyping and Experimentation

AI development involves heavy trial and error (tuning hyperparameters). Jupyter Notebooks allow for **state persistence**.

- **How it works:** When you run a cell, the variables (like a massive dataset of 50,000 images) stay loaded in the computer's memory (RAM).
- **Why it matters:** In a standard script, if you want to change one small setting in your neural network, you often have to re-run the whole script, which might take 10 minutes just to reload the data. In Jupyter, the data stays loaded. You can tweak the model and re-run *only the training block* instantly, speeding up the experimentation loop significantly.

Q3: How does spaCy enhance NLP tasks compared to basic Python string

operations? Basic Python string operations (like `.split()`, `.replace()`, or Regex) treat text as a sequence of characters without understanding the meaning. **spaCy** enhances NLP by treating text as a **linguistic structure**, adding layers of grammatical and semantic understanding.

Here are the specific enhancements:

1. Linguistic Context (POS Tagging & Dependency Parsing):

- a. *Basic Python*: Can split a sentence into words, but doesn't know a "verb" from a "noun."
- b. *spaCy*: Assigns a Part-of-Speech (POS) tag to every word (e.g., knowing that "google" is a verb in "I will google it" but a noun in "Google is a company"). It also maps the grammatical relationships between words (Dependency Parsing), allowing the AI to understand *who* did *what* to *whom*.

2. Named Entity Recognition (NER):

- *Basic Python*: Requires writing hundreds of complex Regex rules to find names or dates, which is brittle and error-prone.
- *spaCy*: Uses pre-trained statistical models to automatically detect and classify real-world objects like Persons, Organizations, Countries, and Dates with high accuracy.

3. Smart Tokenization:

- *Basic Python*: `.split(" ")` fails on punctuation (e.g., "apple," becomes one token) or contractions ("don't").
- *spaCy*: Uses non-destructive tokenization rules to correctly handle punctuation, contractions (splitting "don't" into "do" + "n't"), and special characters, preserving the original text alignment.

4. Semantic Similarity (Word Vectors):

- *Basic Python*: Can only check if string "A" equals string "B".
- *spaCy*: Can measure how *similar* two words are in meaning (e.g., understanding that "dog" and "puppy" are related) using multi-dimensional word vectors

2. Comparative Analysis

- Compare Scikit-learn and TensorFlow in terms of:

Target applications (e.g., classical ML vs. deep learning).

Scikit-learn:

- **Focus**: Classical / Traditional Machine Learning.
- **Best For**: Working with structured (tabular) data, like spreadsheets or SQL databases. It excels at algorithms where you can easily explain the logic, such as

Linear Regression, Random Forests, Support Vector Machines (SVM), and K-Means Clustering.

- **AgTech Example:** Predicting next week's crop yield based on a spreadsheet of historical rainfall and fertilizer amounts.

TensorFlow:

- **Focus:** Deep Learning and Neural Networks.
- **Best For:** Working with unstructured data, such as images, audio, and large blocks of text. It is built to perform complex math (matrix multiplication) on massive datasets using GPUs.
- **AgTech Example:** Recognizing a specific type of pest in a photo taken by a drone or an automated tractor.

Ease of use for beginners.

Scikit-learn:

- **Verdict: High (Easiest).**
- **Why:** It has a legendary "Unified API." Almost every algorithm follows the exact same three steps:
 - `model = Algorithm()` (Create)
 - `model.fit(data)` (Train)
 - `model.predict(new_data)` (Predict)
- This consistency allows beginners to swap out a complex algorithm for a simple one by changing just one line of code.

TensorFlow:

- **Verdict: Medium.**
- **Why:** While it was historically very difficult, the addition of the **Keras** high-level API has made it much friendlier. However, you still need to understand deep learning concepts (like layers, activation functions, and optimizers) to use it effectively. Debugging errors in TensorFlow can also be more cryptic than in Scikit-learn.

Community support.

Scikit-learn:

- **Type of Support:** Mature and stable.
- **Ecosystem:** It is the "default" for data science in Python. Because it has been around for so long, nearly every error you encounter has already been solved on StackOverflow. It integrates perfectly with other standard tools like Pandas and Matplotlib.

TensorFlow:

- **Type of Support:** Massive and industrial.
- **Ecosystem:** Backed by Google, it has a sprawling ecosystem. It offers support that Scikit-learn does not, such as tools for mobile apps (TFLite), web browsers (TF.js), and massive enterprise cloud pipelines (TFX). If you want to deploy a model to a phone, the TensorFlow community has the best resources.